BLE Tag2 White Paper

A concise technical white paper on the manufacturer payload observed on `BLE Tag2` advertisements. The emphasis is the payload structure, the meaning of each byte, and the live-sniff evidence that shows byte 9 behaving like a firmware-side latch.

Key conclusion: the decompiled app uses generic field names, but live traces show a more useful interpretation. Byte 0 behaves like a button press counter, byte 1 behaves like battery percentage, byte 2 is fast polling, and byte 9 is a transient mode/event latch.

Observed payload length	`10 bytes`
Bytes decoded by app	`0..8`
Byte ignored by app	`9`
Most important inference	`Byte 9 is a mode/event latch`
Live decay window	`8-10 s`

What the app does and does not do

The decompiled `CaBLE 4.9` app reads the manufacturer data from the advertising packet and maps the first few bytes into model fields. It does not contain any code path that resets byte 9. The 1-second refresh timer seen in the UI only redraws the list; it does not affect the advertised payload on air.

The app's internal names are generic and can be misleading if taken literally. For `BLE Tag2`, the live traces are the source of truth for the actual byte meaning.

Payload Structure

The manufacturer payload observed in the captures is 10 bytes long. Multi-byte fields are little-endian. Bytes 3..8 form the extended counters, while byte 9 is a separate state latch that the app ignores.

Offset	Size	Meaning	Confidence
`0`	`1`	Button press counter	High
`1`	`1`	Battery percentage	High
`2`	`1`	Fast polling flag	High
`3..4`	`2`	upTimeFast	High
`5..6`	`2`	upTimeSlow	High
`7..8`	`2`	connectionCounter	High
`9`	`1`	modeLatch	Medium-high

Interpretation of byte 9

Byte 9 is the most important unresolved field. In multiple live traces it flipped from 0 to 1 shortly after a state transition correlated with a press, then returned to 0 after roughly 8 to 10 seconds. That pattern fits a mode/event latch much better than a history counter or checksum.

The strongest practical reading is: byte 9 = 1 during a transient active window, and byte 9 = 0 once the tag returns to its slower or inactive state.

Trace	Observed payload	What changed
Press correlation	`06 64 01 0a 00 5f 00 00 00 01`	Byte 0 incremented, byte 9 became 1
Decay back	`06 64 01 0a 00 5f 00 00 00 00`	Byte 9 returned to 0 after about 9.588 s
Stable inactive	`00 61 01 62 00 4b 03 00 00 00`	Byte 9 remained 0

Practical Takeaway

If you are decoding `BLE Tag2` payloads for tooling or reverse engineering, the safest operational view is: byte 0 = button press counter, byte 1 = battery percentage, byte 2 = fast polling, bytes 3..8 = extended counters, and byte 9 = firmware-side latch. That is the model that best matches the live captures we collected.

The app's source names should be treated as implementation labels rather than final semantic truth. For this device family, the observed on-air behavior is the source of truth.

Sources: `cable_ble_tag2_reverse_notes.md`, `ble_tag2_decoder_spec.md`, and `ble_tag2_decoder.py`.